

Types, Variables and Input

L12 - Input

Department of Computer Science
University of Pretoria

18 March 2024

- In C++, a **variable** is a named storage location in the computer's memory that can hold a value of a particular data type.
- Variables allow programmers to store and manipulate data within a program.
- In C++, all the variables **must** be declared before you can use them.

Examples of C++ code with variables

```
1 #include <iostream>
2 int main() {
3     //declaration of variables
4     int age;
5     float temperature;
6     char grade;
7     //assigning values to variables
8     age = 30;
9     temperature = 72.8;
10    grade = 'C';
11    return 0;
12 }
```

Rules for declaring variables in C++:

- Variable names in C++ **must** start with a letter (a-z or A-Z) or an underscore character. They can be followed by letters, digits (0-9), or underscores.
- Variable names **cannot** be the same as C++ keywords or reserved words.

- Variable names **cannot** contain spaces or most special characters except underscore.
- C++ is **case-sensitive**, so variable names with uppercase and lowercase letters are treated differently.

C++ Constants:

- In C++, constants are like variables, but their values cannot be modified once they are initialized.
- Constants are useful for **representing fixed values** in your program that should not change during its execution.

C++ Constants:

- In C++, you can define constants using the **define** preprocessor method.
- Constants defined with **define** method are processed by the preprocessor before the actual compilation of the code.

define method implementation

```
1 #include <iostream>
2 #define PI 3.14
3 #define MAX_SIZE 100
4 using namespace std;
5 int main() {
6     int RADIUS = 10;
7     cout << "The area of a circle with radius 10cm is : -" << endl;
8     cout << PI * RADIUS * RADIUS;
9
10    return 0;
11 }
```


- In C++, you can also define constants using the **const** keyword.
- Constants declared with **const** are variables whose values cannot be changed once they are initialized.

const keyword implementation

```
1 #include <iostream>
2 const float PI = 3.1415;
3 using namespace std;
4 int main() {
5     const int RADIUS = 10;
6     cout << "The area of a circle with radius 10cm is :-"<<endl;
7     cout << PI * RADIUS * RADIUS;
8
9     return 0;
10 }
```

Basic Maths operators in C++

- In C++, there are several mathematical operators used to perform arithmetic operations on numeric data types.
- These operators allow you to perform addition, subtraction, multiplication, division, and more.

Basic Maths operators in C++

- Addition (+), Subtraction (-), Division (/) and Multiplication (*)
- Modulus , Increment (++), Decrement (--), Compound (+=, -=, *=, /=,)

Basic Maths operators Code Example

```
1 #include <iostream>
2
3 int main() {
4     int a = 10, b = 3, result;
5     double quotient;
6     result = a + b;
7     std::cout << "Addition:-" << result << std::endl;
8     result = a - b;
9     std::cout << "Subtraction:-" << result << std::endl;
10    result = a * b;
11    std::cout << "Multiplication:-" << result << std::endl;
12    quotient = static_cast<double>(a) / b;
13    std::cout << "Division:-" << quotient << std::endl;
14    result = a % b;
15    std::cout << "Modulus:-" << result << std::endl;
16    int x = 5, y = 10;
17    std::cout << "Increment:-" << ++x << std::endl;
18    std::cout << "Decrement:-" << --y << std::endl;
19
20    return 0;
21 }
```

Logical Operators

- In C++, logical operators are used to perform logical operations on boolean expressions.
- These operators allow you to combine multiple conditions and evaluate them to produce a single boolean result.

Logical operators used in C++:

- **Logical AND** `&&`: The logical AND operator returns true if both operands are true, otherwise, it returns false.
- **Logical OR** `||`: The logical OR operator returns true if at least one of the operands is true, otherwise, it returns false.

Logical operators used in C++:

- **Logical NOT (!):** The logical NOT operator is a unary operator that reverses the logical state of its operand. If the operand is true, it returns false, and if the operand is false, it returns true.

Logical Operators Code Example

```
1 #include <iostream>
2 int main() {
3     // Example variables
4     int x = 3, y = 6;
5     // Logical AND (&&)
6     if (x > 0 && y > 0) {
7         std::cout << "Both x and y are positive." << std::endl;
8     }
9     // Logical OR (||)
10    if (x == 0 || y == 6) {
11        std::cout << "Either x or y is zero or six." << std::endl;
12    }
13    // Logical NOT (!)
14    if (!x == 0) {
15        std::cout << "x is not equal to zero" << std::endl;
16    }
17    return 0;
18 }
```

- In C++, **cin** is an object defined in the standard library's **iostream** header.
- **cin** is an abbreviation for **character input**.
- It is used for standard input, typically to read data from the standard input stream, which is usually the keyboard.

- cin is primarily used with the **extraction operator** `>>` to read input from the user and store it into variables.
- It can read various types of data, such as integers, floating-point numbers, characters, and strings.

Cin Code Example

```
1 #include <iostream>
2 using namespace std;
3 int main() {
4     //declaring variables
5     int x, y;
6     int sum;
7     cout << "Type-First-No:-";
8     //inputting variables
9     cin >> x;
10    cout << "Type-Second-No:-";
11    cin >> y;
12    //calculating the sum
13    sum = x + y;
14    cout << "The-sum-is:-" << sum;
15    return 0;
16 }
```

Class Question

Write a C++ program that can be used to swap two numbers. The program must initially accept input of two numbers from the user, then swap the two numbers and outputs the result. Below is sample output of expected program in operation.

Enter 1st number: 50

Enter 2nd number: 22

1st number after swap: 22

2nd number after swap : 50

Question Solution

```
1 #include <iostream>
2 using namespace std;
3 int main()
4 {
5     //declaring variables
6     int num1, num2, temp;
7     //inputting the two numbers
8     cout << "Enter-1st-number:";
9     cin >> num1 ;
10    cout << "Enter-2nd-number:";
11    cin >> num2;
12    //swapping the two numbers
13    temp = num2;
14    num2 = num1;
15    num1 = temp;
16    //printing the results
17    cout << "1st-number-after-swap:"<< num1 <<endl ;
18    cout << "1st-number-after-swap:"<< num2 <<endl ;
19    return 0;
20 }
```

- C++ supports formatting of various data types using the **iostream** and **iomanip** libraries.
- In C++, **setw** and **setprecision** are manipulators provided by the `< iomanip >` header.
- They are often used together to format output, especially when displaying tabular data or numeric values with specific formatting requirements.

setw manipulator

- This manipulator sets the width of the field in which subsequent output is displayed.
- It is often used to align output in columns.

setw code implementation

```
1 #include <iostream>
2 #include <iomanip>
3 using namespace std;
4 int main() {
5     //declaring required variables
6     int num1,num2;
7     num1 = 10;
8     num2 = 20;
9     //Before setting width
10    cout<< num1 << num2 <<endl;
11    //After setting width
12    cout << setw (5) << num1 << setw (5) << num2 << endl;
13    return 0;
14 }
```

- In this example, `setw(5)` sets the width of the field to 5 characters for each output. If the printed value is less than 5 characters wide, the remaining space will be filled with spaces.

setprecision Manipulator

- This manipulator sets the precision of floating-point numbers when they are displayed.
- It determines the number of digits to be displayed after the decimal point.

setprecision code implementation

```
1 #include <iostream>
2 #include <iomanip>
3 using namespace std;
4 int main() {
5     double num1 = 45.3423423;
6     double num2 = 12.43545;
7     //significant figures
8     cout << setprecision(3) << num1 <<endl;
9     cout << setprecision(3) << num2 <<endl;
10    //decimal places
11    cout << fixed << setprecision(3) << num1 <<endl;
12    cout << fixed << setprecision(3) << num2 <<endl;
13    return 0;
14 }
```

- In this example, lines 8 and 9 sets the output to **three** significant figures. Where as in lines 11 and 12 sets the decimal places to three. This is through the use of the **fixed** keyword.